Assignment 3:
Build & Deploy a Full Stack App

Due Week 4 at the start of class.

Goal Build a full-stack app with auth, a database, and an external API. Anyone should be able to sign up and use it.

Overview

You built a tool with Next.js + Tailwind last week. Now we add the backend: Clerk for auth, Supabase for data, and a public API for richer content. Pick an idea that lets users search or browse data from an API and save their favorites. Your classmates should be able to create accounts on your app and use it.

Ideas to Explore (Pick One)

➡️ **Extend Assignment 2: You can add an external API, Supabase, and Clerk to Assignment 2 if you'd like. Or start fresh...**

🎬 Movie/TV Tracker — search movies (OMDB or TMDB API), save to watchlist, mark as watched, rate them.
🍳 Recipe Finder — search recipes (TheMealDB — no key), save favorites, plan weekly meals.
📚 Book Club — search books (Open Library — no key), save to reading list, track status (want to read / reading / finished).
🐉 Pokémon Team Builder — browse Pokémon (PokéAPI — no key), build a team of 6, save multiple teams.
🎮 Game Tracker — search games (RAWG API — free key), track your backlog, mark as played, rate them.
🌍 Travel Bucket List — browse countries (REST Countries — no key), save a travel bucket list, add notes.
🚀 Space Gallery — daily space photos (NASA APOD — free key), save favorites, build a personal gallery with notes.
🎵 Music Discovery — search artists/albums (MusicBrainz — no key), save a listening list, rate albums.
✨ Your Own Idea — any public API + saved user data. If it searches, saves, and has accounts, it counts.

Requirements

- Built with Next.js + Tailwind CSS
- User authentication via Clerk (sign up, log in, sign out)
- Data stored in Supabase, scoped to the logged-in user
- Fetches data from an external API
- Users can search/browse API data and save items to their account
- Users can view their saved items
- Environment variables in .env.local (Supabase + Clerk keys), not hardcoded
- Supabase MCP server configured
- Multiple git commits showing iteration
- Deployed to Vercel with environment variables set
- Live URL works — classmates can create accounts and use it

Steps

1. Pick your idea + API. Choose a public API and an idea that makes sense with it. What data are users searching for? What are they saving?

2. Scaffold + GitHub. Create a Next.js app. Write a CLAUDE.md describing your project and data model. Push to GitHub. Commit after each phase.

3. Set up Supabase + MCP. Create a project, configure MCP (claude mcp add --transport http supabase https://mcp.supabase.com/mcp), have Claude create your tables.

4. Set up Clerk. Create a Clerk app, add keys to .env.local, set up sign-in/sign-up pages.

5. Connect Clerk + Supabase. Clerk dashboard → "Connect with Supabase." Supabase → Auth → Third-Party Auth → Add Clerk.

6. Build it. Search/browse the API, save to Supabase scoped to the logged-in user, display saved items.

7. Deploy. Push to GitHub, deploy on Vercel, add all environment variables.

Stretch Goals

Not required, for students who finish early:

- A public or community view (see what others have saved)
- Remove/unsave items
- Detailed view (click an item for more info from the API)
- Sorting or filtering saved items
- Search history
- Mobile-responsive design

Submission

Deliverables

1. Vercel URL — A live link to your tool.

2. GitHub URL — Public repository with commit history.

3. Reflection Questions

1. Trace a request: a user searches, saves, and views it on their profile. What systems are involved?
2. Why should your app call the external API from the server (API route) instead of directly from the browser?
3. A classmate signs up on your app. What data does Clerk store vs. what does Supabase store? How are they connected?
4. Ask Claude (with MCP) to describe your database. Paste the response. Does it match your mental model?

Example Submission

Assessment

A complete assignment has:

- ☐ A live Vercel URL that works — classmates can sign up
- ☐ Auth via Clerk (sign up, log in, sign out)
- ☐ Data from an external API, saved to Supabase per user
- ☐ A GitHub repo with multiple commits showing iteration
- ☐ Thoughtful answers to the 4 reflection questions

You'll be assessed on whether you used the full stack, iterated with intention, and built something your classmates can actually use.

Notes

Refer to Week 3 slides and Week 3 Companion in Google Classroom for more detailed steps. Commit and push as you go — it lets you rollback when needed. Having Claude do the commits speeds things up.

Explore Claude Code

CLAUDE.md — Create one in your project root with what you're building and style preferences.

Plan mode — Shift+Tab to plan before building. Read the plan, then let Claude build.

Supabase MCP — claude mcp add --transport http supabase https://mcp.supabase.com/mcp

Prompting — Try vague vs. specific prompts. Iterate: "bolder typography," "more whitespace."

Commands — /context · /compact · /clear · /diff · Esc Esc (undo) - https://code.claude.com/docs/en/commands

Questions? Reach out on Slack or come to office hours.